



Nombre:

Grupo:

Pregunta 1:

Contestar V/F las siguientes cuestiones, teniendo en cuenta que una respuesta incorrecta invalida una correcta.

[] Un vector dinámico es una estructura de datos básica que puede ser utilizada en la implementación de asociaciones y composiciones múltiples cuando no hay restricciones o necesidades especiales en las mismas.

[] En una lista simplemente enlazada el borrado del último elemento requiere tiempo lineal incluso si se mantiene permanentemente un puntero al mismo.

[] Una lista doblemente enlazada permite realizar búsquedas binarias en tiempo $O(\log n)$ si los datos se encuentran ordenados.

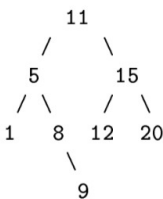
[] Se ha utilizado la siguiente estructura de datos para implementar un editor de texto interactivo: $list<vector<char>> texto$. La implementación de la operación *pulsarTecla(char c)* sería la siguiente: $i->insert(i->begin() + pos, c)$, siendo i un iterador de la lista que apunta a la línea donde se encuentra el cursor y pos un entero positivo que indica la posición del cursor en dicha línea.

[] Tanto árboles binarios de búsqueda como árboles AVL son sensibles al orden de introducción de un conjunto de datos, pudiendo haber una diferencia grande de rendimiento entre una situación y otra en ambos casos.

[] La siguiente tabla referente a la eficiencia de las distintas implementaciones de una cola con prioridad es correcta (n es el número de datos y p el número de valores de prioridad distintos):

Implementación	push	pop
vector de listas	$O(1)$	$O(p)$
lista de listas	$O(p)$	$O(1)$
heap	$O(\log n)$	$O(\log n)$

[] Dado el siguiente árbol AVL, la inserción de 10 requiere una rotación doble a derecha.



[] Los agrupamientos secundarios se producen cuando claves que son dispersadas a posiciones diferentes siguen la misma secuencia de búsqueda para localizar una posición disponible.

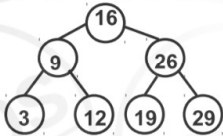
[] Un quadtree puede guardarse en un vector de forma similar a un heap. Dado un nodo no hoja situado en la posición i del vector, sus nodos hijo estarían localizados en las posiciones $i * 4 + h$, siendo $h = \{1, 2, 3, 4\}$ el número de hijo.

[] Al igual que los índices simples, los árboles B se mantienen permanentemente en memoria. Su ventaja respecto a los índices es la mayor eficiencia en la búsqueda de una clave: $O(\log_2 n)$ de los índices frente a $O(\log_m n)$ de los árboles B, siendo m el orden del mismo y siempre mucho mayor que 2.

Pregunta 2:

Realizar sobre la siguiente configuración de un árbol AVL las siguientes operaciones de inserción (I) y borrado (B) en este orden:

I(17), I(18), I(20), I(22), B(29)



Pregunta 3:

- Implementar la función $ListaDEnlazada<T>::operator+(ListaDEnlazada<T> \&l)$; que concatene los elementos de la lista doblemente enlazada con los de otra que entra por parámetro.
- Implementar la función $bool DispCerrada<T>::buscar(long clave, T \&dato)$; que realiza la búsqueda de un dato a partir de su clave en la tabla de dispersión cerrada con exploración lineal que está definida a continuación. El dato deberá ser devuelto en el parámetro *dato*, y el retorno de la función será *true* si la clave existe o *false* en caso contrario. Codificar la clave con valores negativos para representar los estados de borrado y disponible.

```
template<class T>
class Entrada {
```



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional

```
    long clave;
    T dato;
    Entrada(long aClave, T aDato): clave(aClave), dato(aDato) {}
};
template<class T>
class DispCerrada {
    vector<Entrada> tabla;
    long fDisp(long clave);
public:
    .....
    buscar (long clave, T &dato);
};
```

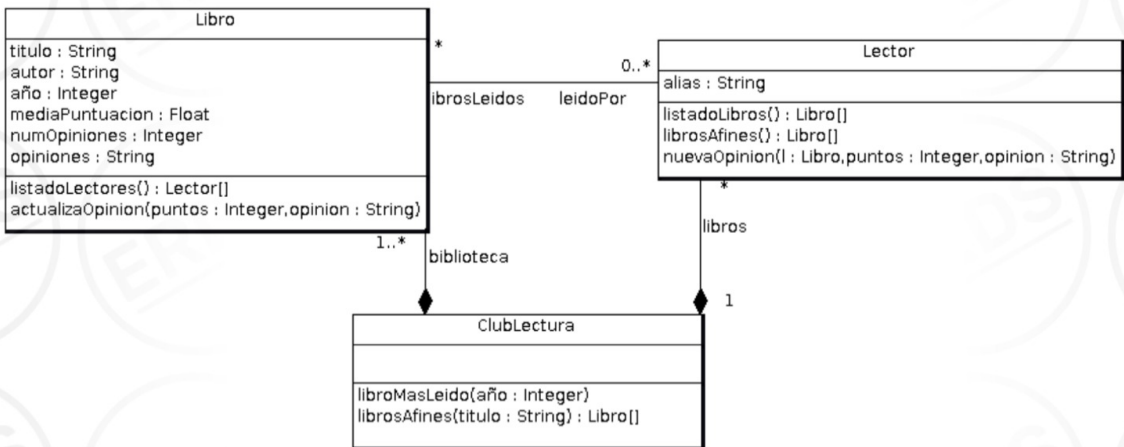
Pregunta 4:

Se ha creado un portal en Internet a modo de club de lectura en el que se guardan miles de referencias a libros y las opiniones de los lectores. Cada libro tiene una puntuación media que le dan todos los lectores, lo que sirve de recomendación para saber la opinión de otros lectores hacia ese libro.

Un *libro* tiene título, autor, año de edición, un número de opiniones que indica el número de lecturas de ese libro y una media de valoración de los lectores (entre 0 y 10). También tiene un campo tipo String que almacena paulatinamente las opiniones y comentarios de los usuarios. La función *listadoLectores()* devuelve los lectores que han leído ese libro y han emitido opinión sobre él y *actualizaOpinion()* actualiza la puntuación media y añade una nueva opinión sobre el libro.

Un *Lector* tiene un alias en el sistema y emite cada vez que lee un libro una valoración sobre ese libro con *nuevaOpinion()*, lo que actualiza el estado de ese libro en el sistema. Devuelve el listado de todos los libros que ha leído con *listadoLibros()* y además consigue conocer los 50 libros mejor valorados de todos los lectores que también han leído alguno sus libros con la función *librosAfines()*.

El club de lectura gestiona las siguientes consultas: el libro más leído que se editó un año concreto o los libros más compatibles con un título (librosAfines), es decir, la lista de los 50 libros más valorados por los lectores que leyeron ese libro en concreto.



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional